Backend Team Game Plan

Mission

Turn AgentHarbor's backend into a demo-ready control plane that feels live, reliable, and intentional.

The current repository already has a strong MVP:

- `apps/control-node` exposes enrollment, heartbeat, telemetry, list, and stats APIs.
- `apps/runner` can enroll, send heartbeats, emit telemetry, and run a simple demo scenario.
- `packages/shared` defines the telemetry contracts.
- `packages/sdk` gives the runner and future clients a transport boundary.

The backend team's job is not to build orchestration in three and a half weeks.
The backend team's job is to make the platform visibly excellent at telemetry ingestion, fleet state, analytics, and real-time updates.

Demo Story To Build Toward

By demo day, the backend should support this flow without fragile hand-waving:

1. Start the control node and dashboard.
2. Enroll multiple runners that represent different AI agents.
3. Generate multiple sessions at once, with different outcomes.
4. Stream those updates into the dashboard live.
5. Show filters, analytics, and failure detection working immediately.
6. Drill into a failed session and explain why it failed.
7. Optionally revoke a runner token and show that the runner stops reporting.

If the backend supports that story, the frontend can make it look impressive.

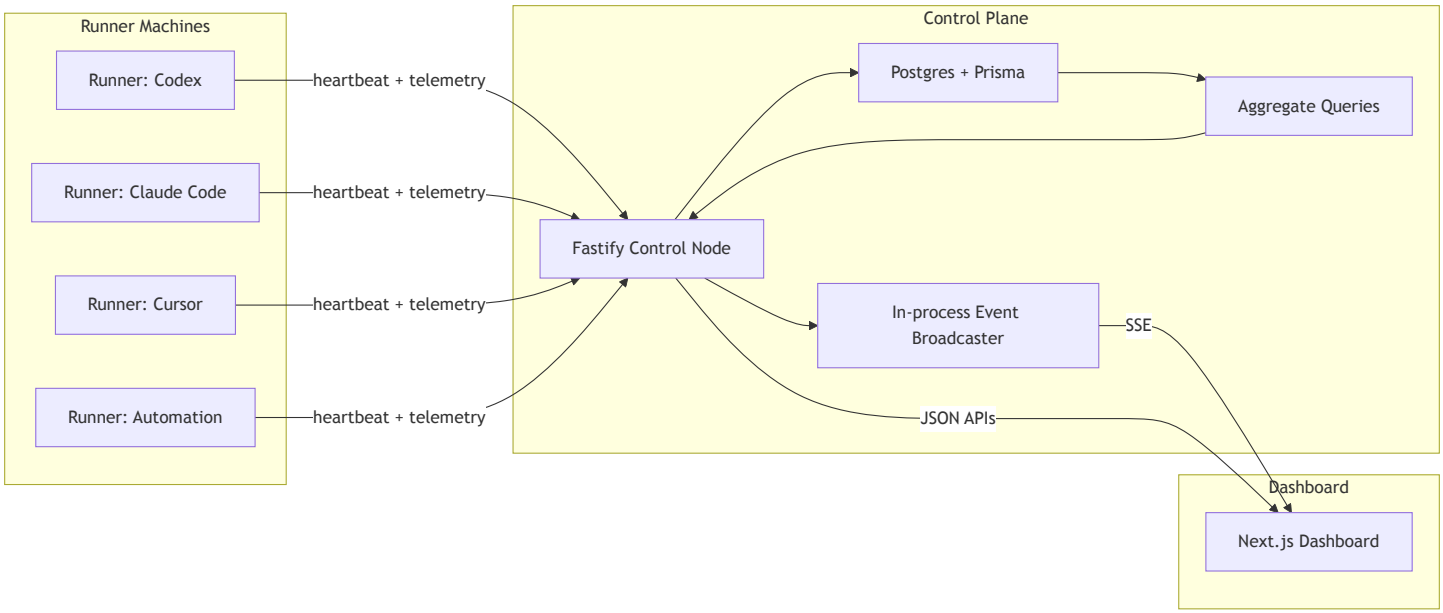
Recommended Team Split

With five students total and a two-team split, a practical backend split is three students:

Student	Primary Ownership	Secondary Ownership	Core Files
Backend A	Runner automation and demo scenarios	Seed scripts	`apps/runner/src/index.ts` , `packages/sdk/src/client.ts`
Backend B	Control-node filters, analytics, SSE	Shared contracts	`apps/control-node/src/routes/v1.ts` , `packages/shared/src/telemetry.ts`
Backend C	Prisma schema, testing, auth hardening	Dev tooling	`apps/control-node/prisma/schema.prisma` , `apps/control-node/src/lib/auth.ts`

Do not isolate them completely. Every backend deliverable should help a frontend page become more believable.

System Target



What Already Exists

The current control-node routes are in `apps/control-node/src/routes/v1.ts` .
The current runner behavior is in `apps/runner/src/index.ts` .
The current telemetry schema is in `packages/shared/src/telemetry.ts` .

That matters because the backend plan should extend these seams instead of replacing them.

Backend Deliverables

1. Multi-Runner Demo Engine

The current `demo` command emits one linear session at a time. That is not enough for a strong demo.

Build a richer demo runner that can simulate:

- a healthy session that completes successfully
- a long-running session
- a failed session
- multiple runners reporting in parallel
- a noisy runner that emits many events

The easiest way to do this is to keep the existing CLI and add scenario-based commands.

Example command shape:

```
cd apps/runner
node dist/index.js demo --scenario mixed-fleet --runners 4 --cycles 6 --interval-ms 120
```

Suggested scenarios:

- `happy-path`
- `mixed-fleet`
- `failure-burst`
- `recovery`

Suggested implementation sketch:

```typescript
type DemoScenarioName = "happy-path" | "mixed-fleet" | "failure-burst" | "recovery";

interface DemoScenarioContext {
  runnerName: string;
  agentType: "codex" | "claude-code" | "cursor" | "automation";
  cycle: number;
}

function buildScenarioEvents(name: DemoScenarioName, context: DemoScenarioContext) {
  if (name === "failure-burst") {
    return [
      buildEvent("agent.session.started", {
        agentType: context.agentType,
        sessionKey: `${context.runnerName}-failure-${context.cycle}`,
        summary: "Accepted task and initialized workspace context.",
        category: "session",
        status: "running",
      }),
      buildEvent("agent.prompt.executed", {
        agentType: context.agentType,
        sessionKey: `${context.runnerName}-failure-${context.cycle}`,
        summary: "Encountered repeated build failures while applying the patch.",
        category: "build",
        status: "blocked",
        tokenUsage: 980,
        filesTouchedCount: 4,
      }),
      buildEvent("agent.session.failed", {
        agentType: context.agentType,
        sessionKey: `${context.runnerName}-failure-${context.cycle}`,
        summary: "Session failed after repeated build and test errors.",
        category: "failure",
        status: "failed",
        durationMs: 42_000,
        tokenUsage: 1_450,
        filesTouchedCount: 6,
      }),
    ];
  }

  return [];
}
```

Definition of done:

- One command can create enough realistic traffic to populate the dashboard for the entire presentation.
- At least one scenario produces failures on purpose.
- At least one scenario creates concurrent runner activity.

2. Automatic Heartbeat Scheduling

Right now, `heartbeat` is a manual CLI action. For the demo, a manual heartbeat is too brittle.

Add a background heartbeat mode in the runner:

- send heartbeat every 10 to 15 seconds
- include active session count
- retry with exponential backoff
- stop gracefully on process exit

Implementation sketch:

```
async function startHeartbeatLoop(client: AgentHarborClient, getActiveSessionCount: ()
  let stopped = false;
  let delayMs = 10_000;

  const tick = async () => {
    while (!stopped) {
      try {
        await client.sendHeartbeat({
          timestamp: new Date().toISOString(),
          activeSessionCount: getActiveSessionCount(),
          metadata: { mode: "daemon" },
        });
        delayMs = 10_000;
      } catch (error) {
        delayMs = Math.min(delayMs * 2, 60_000);
      }

      await sleep(delayMs);
    }
  };

  void tick();

  return () => {
    stopped = true;
  };
}
```

Definition of done:

- A runner can stay online during the entire demo without manual intervention.
- If the control node is restarted, the runner resumes reporting.

3. Filterable Read APIs

The current list routes only support `limit`.
That is fine for an MVP and not enough for a polished demo.

Add filters to:

- `GET /v1/runners`
- `GET /v1/sessions`

- `GET /v1/events`

Minimum filter set:

- `status`
- `agentType`
- `runnerId`
- `sessionId`
- `eventType`
- `since`
- `search`
- `label`
- `limit`

Suggested query schema pattern:

```
const sessionQuerySchema = z.object({
  limit: z.coerce.number().int().positive().max(100).optional(),
  status: z.enum(sessionStatuses).optional(),
  agentType: z.enum(agentTypes).optional(),
  runnerId: z.string().optional(),
  since: z.string().datetime().optional(),
  search: z.string().trim().min(1).optional(),
});
```

Suggested Prisma mapping pattern:

```
const where: Prisma.AgentSessionWhereInput = {
  ...(query.status ? { status: query.status } : {}),
  ...(query.agentType ? { agentType: query.agentType } : {}),
  ...(query.runnerId ? { runnerId: query.runnerId } : {}),
  ...(query.since ? { startedAt: { gte: new Date(query.since) } } : {}),
  ...(query.search
    ? {
        OR: [
          { sessionKey: { contains: query.search, mode: "insensitive" } },
          { summary: { contains: query.search, mode: "insensitive" } },
        ],
      }
    : {}),
};
```

Definition of done:

- The frontend can drive the entire demo from query parameters alone.
- Every filter used in the UI maps to a real API query, not client-side fake filtering.

4. Runner Labels And Grouping

The roadmap already mentions runner labels and grouping.
This is a good senior-design-sized feature because it improves both the data model and the UI.

Pragmatic schema change:

```
model Runner {
  id              String           @id @default(cuid())
  name            String
  machineName     String
  status          RunnerStatus     @default(enrolled)
  labels          String[]         @default([])
  environment     String?
  createdAt       DateTime         @default(now())
  updatedAt       DateTime         @updatedAt
  lastSeenAt      DateTime?
  machineId       String
  machine         Machine          @relation(fields: [machineId], references: [id], onD
  sessions        AgentSession[]
  telemetryEvents TelemetryEvent[]
  tokens          RunnerToken[]
}
```

Suggested label values for the demo:

- `demo`
- `frontend`
- `backend`
- `macos`
- `lab-machine`
- `student-team-a`
- `student-team-b`

Definition of done:

- Runners can be filtered and grouped by labels.

- Demo scenarios assign labels consistently.

5. Real-Time Event Streaming

This is the single most important backend feature for the demo.
If the dashboard updates live, the system feels real.

Use Server-Sent Events first.
Do not spend the schedule on WebSockets unless SSE becomes a blocker.

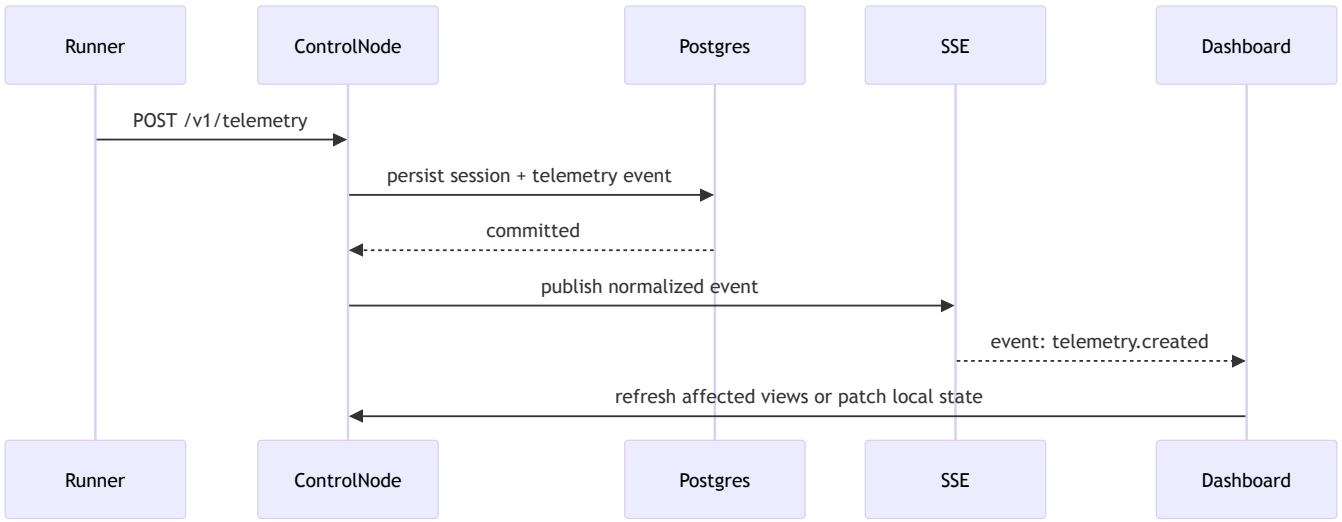
Recommended endpoint:

- `GET /v1/stream/events`

Broadcast at least:

- runner heartbeats
- session state changes
- telemetry events
- aggregate stat refresh hints

High-level design:



Simple broadcaster sketch:

```typescript
type StreamListener = (payload: unknown) => void;

const listeners = new Set<StreamListener>();

export function publishStreamEvent(payload: unknown) {
  for (const listener of listeners) {
    listener(payload);
  }
}

export function subscribeStream(listener: StreamListener) {
  listeners.add(listener);
  return () => listeners.delete(listener);
}
```

Route sketch:

```typescript
app.get("/v1/stream/events", async (request, reply) => {
  reply.raw.writeHead(200, {
    "Content-Type": "text/event-stream",
    "Cache-Control": "no-cache",
    Connection: "keep-alive",
  });

  const unsubscribe = subscribeStream((payload) => {
    reply.raw.write(`event: telemetry.created\n`);
    reply.raw.write(`data: ${JSON.stringify(payload)}\n\n`);
  });

  request.raw.on("close", () => {
    unsubscribe();
  });
});
```

Definition of done:

- The frontend can subscribe once and receive new telemetry in real time.
- Reconnecting the page does not crash the stream.
- The stream works during the actual presentation on local infrastructure.

6. Analytics Endpoints

The current `/v1/stats` route is a good start.
Expand it so the dashboard can show trends and not just totals.

Recommended endpoints:

- `GET /v1/stats`
- `GET /v1/analytics/agent-types`
- `GET /v1/analytics/failures`
- `GET /v1/analytics/runners/activity`
- `GET /v1/analytics/events/timeseries`

Minimum useful outputs:

- sessions grouped by agent type
- failures grouped by category
- event count by minute or five-minute bucket
- top runners by session volume

Example response shape:

```
{
  "points": [
    { "bucketStart": "2026-04-20T18:00:00.000Z", "count": 5 },
    { "bucketStart": "2026-04-20T18:05:00.000Z", "count": 11 },
    { "bucketStart": "2026-04-20T18:10:00.000Z", "count": 8 }
  ]
}
```

Definition of done:

- The frontend can render at least three distinct visualizations from real backend aggregates.
- One chart must visibly spike during the live demo when events arrive.

7. Session Failure Explanations

The frontend can only tell a good story if the backend preserves enough structure.

Standardize failure categories in telemetry payloads.
Recommended categories:

- `build`
- `test`
- `auth`
- `network`
- `timeout`
- `human-approval`
- `unknown`

You do not need a full taxonomy.
You do need consistency so the UI can show useful labels.

Suggested shared-contract addition:

```
export const eventCategories = [
  "session",
  "planning",
  "implementation",
  "build",
  "test",
  "failure",
  "network",
  "auth",
] as const;
```

Definition of done:

- Failed sessions carry meaningful categories and summaries.
- The frontend can explain why a session failed without inventing text.

8. Token Revocation Stretch Goal

Only do this after the live demo core is stable.

Stretch endpoint:

- `POST /v1/runners/:id/revoke-tokens`

Minimal behavior:

- mark active tokens as revoked
- future heartbeats and telemetry return `401`
- runner status eventually falls offline

This is a strong demo add-on because it shows security posture, but it is not core.

Test And Demo Tooling

The backend team should create a small, repeatable demo harness.

Required scripts:

- database reset or seed command
- one command to start demo traffic
- one command to simulate a failure-heavy burst

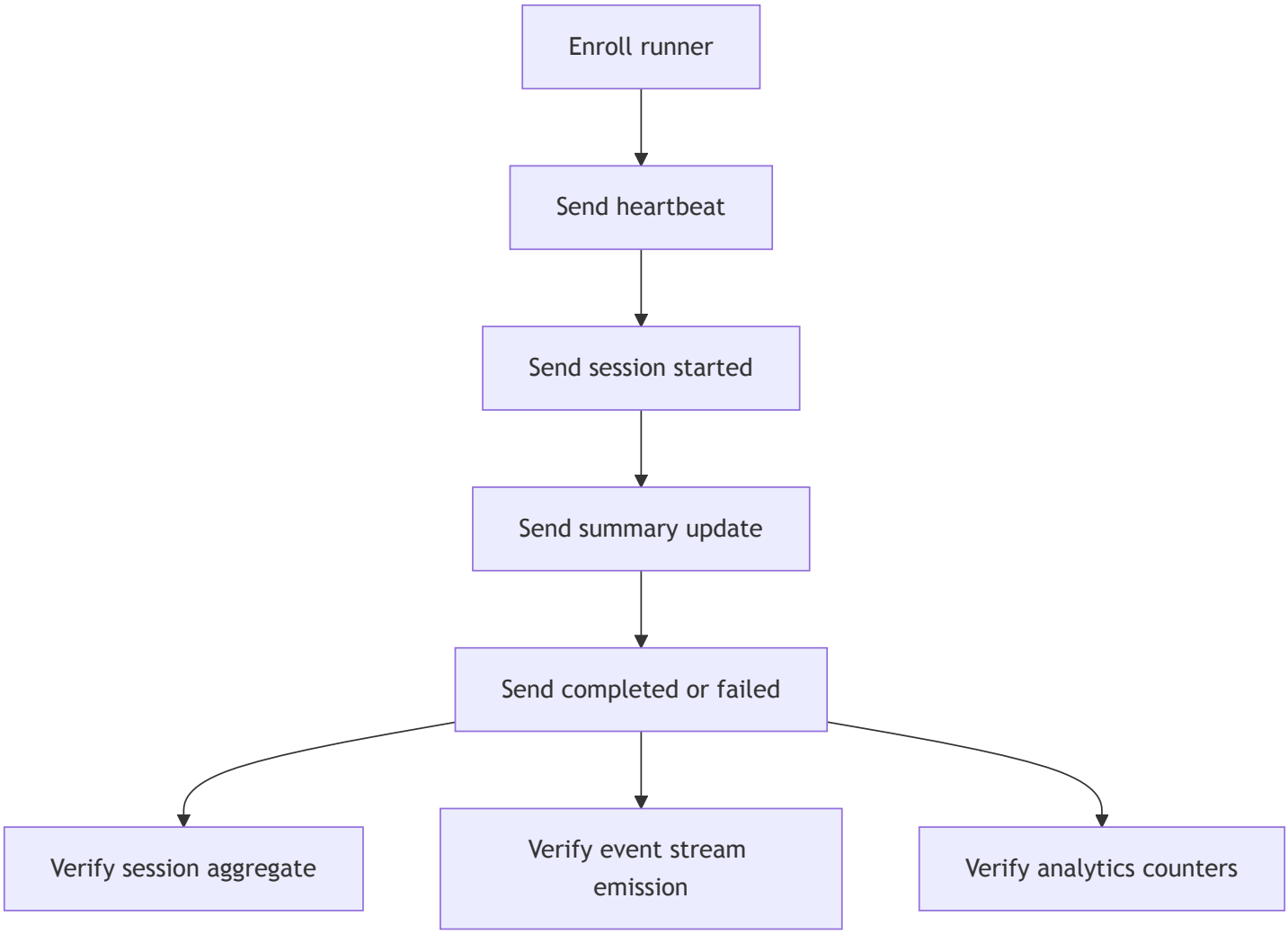
Suggested local commands:

```
docker compose up -d postgres
DATABASE_URL=postgresql://agentharbor:agentharbor@localhost:5432/agentharbor pnpm db:pu
pnpm dev:control
pnpm dev:dashboard
pnpm --filter @agentharbor/runner build
cd apps/runner && node dist/index.js demo --scenario mixed-fleet --runners 4 --cycles 6
```

Testing priorities:

1. enrollment flow
2. heartbeat auth and status update
3. telemetry ingestion and session upsert behavior
4. filter query correctness
5. analytics endpoint correctness
6. SSE event delivery

Recommended backend test matrix:



Implementation Phases

This section translates the backend plan into the order the team should actually execute it.

The goal is to reduce thrash.
Do not build these features in random order.
Build them so each phase unlocks the next one and gives the frontend team something stable to integrate.

Phase 0: Align The Team Around One Backend Story

Objective:

- make sure all three backend students are building toward the same demo behavior

Steps:

1. Read `apps/control-node/src/routes/v1.ts`, `apps/runner/src/index.ts`, and `packages/shared/src/telemetry.ts` together.
2. Decide which demo scenarios are mandatory for presentation day.
3. Decide the telemetry fields that the frontend will rely on for cards, alerts, and timelines.
4. Assign ownership so one student owns runner simulation, one owns control-node APIs, and one owns schema/testing.
5. Write down the exact API additions and schema changes before anyone starts coding.

Outputs:

- agreed list of demo scenarios
- agreed list of query parameters
- agreed event categories and status vocabulary
- student ownership map

Frontend handoff:

- send the frontend team a one-page contract summary for sessions, events, filters, and failure categories

Phase 1: Lock Shared Contracts And Schema Changes

Objective:

- stabilize the data model before building UI-dependent backend features

Steps:

1. Extend `packages/shared/src/telemetry.ts` with any new enums, filters, categories, or response schemas.
2. Update `apps/control-node/prisma/schema.prisma` for labels, environment fields, or any new indexes needed for queries.
3. Run Prisma generation and database push locally.
4. Update response shapes in `apps/control-node/src/routes/v1.ts` so every list route returns the fields the frontend needs.
5. Verify that the runner and dashboard can still compile against the new contracts.

Recommended checklist:

- add runner labels if included in scope

- add failure categories if included in scope
- add indexes for `status`, `startedAt`, `createdAt`, and label-driven queries
- keep response shapes backwards compatible where possible

Outputs:

- stable shared types
- updated Prisma schema
- stable field list for frontend integration

Definition of done:

- no one on the frontend team is guessing payload shapes anymore

Phase 2: Build The Demo Runner And Heartbeat Automation

Objective:

- create realistic traffic that makes the dashboard worth looking at

Steps:

1. Expand the runner CLI so it supports named scenarios instead of one generic demo.
2. Add support for multiple simulated runners and multiple agent types.
3. Add automatic heartbeat scheduling with retry and graceful shutdown.
4. Make sure each scenario emits realistic session lifecycles with useful summaries.
5. Add at least one failure scenario with deliberate failure categories and status values.
6. Validate that running the scenario fills the database with the expected sessions and events.

Recommended scenario build order:

1. `happy-path`
2. `mixed-fleet`
3. `failure-burst`
4. `recovery`

Verification steps:

```
pnpm --filter @agentharbor/runner build
cd apps/runner
node dist/index.js demo --scenario happy-path --cycles 2
node dist/index.js demo --scenario mixed-fleet --runners 4 --cycles 6
```

Outputs:

- scenario-capable runner CLI
- automatic heartbeat loop
- repeatable success and failure traffic

Frontend handoff:

- provide exact commands the frontend team can run to populate the dashboard while building UI

Phase 3: Implement Filterable Read APIs

Objective:

- make the dashboard truly interactive instead of client-side pretending

Steps:

1. Add dedicated query schemas for runners, sessions, and events routes.
2. Implement Prisma `where` clauses for each supported filter.
3. Add filtering for `status`, `agentType`, `runnerId`, `eventType`, `since`, `search`, and `label`.
4. Return results in a stable order so the UI does not flicker unpredictably.
5. Validate that each filter can be exercised directly from the browser or curl.

Recommended validation examples:

```
curl -k "https://localhost:8443/v1/sessions?status=failed&agentType=codex&limit=5"
curl -k "https://localhost:8443/v1/events?eventType=agent.session.failed&limit=10"
curl -k "https://localhost:8443/v1/runners?label=demo&status=online"
```

Outputs:

- filterable list endpoints
- search-driven and label-driven backend queries
- stable sorting for UI surfaces

Definition of done:

- every dashboard control planned by the frontend team has a real backend query to call

Phase 4: Add Live Streaming And Analytics

Objective:

- make the system feel real in motion

Steps:

1. Add an in-process broadcaster or equivalent publish mechanism to the control node.
2. Emit stream events after telemetry and heartbeat writes commit successfully.
3. Add an SSE endpoint for the dashboard to subscribe to.
4. Build aggregate analytics endpoints for agent type distribution, event volume, runner activity, and failure categories.
5. Verify that live traffic causes both the event stream and analytics surfaces to move.
6. Document which stream event types the frontend should listen for.

Recommended implementation order:

1. event broadcaster
2. SSE route
3. telemetry publish path
4. stats refresh hints
5. analytics endpoints

Outputs:

- working SSE endpoint
- analytics routes for charts
- live event messages tied to real writes

Frontend handoff:

- provide sample SSE payloads and analytics responses so the UI can wire against stable shapes

Phase 5: Build The Demo Harness And Test The Critical Paths

Objective:

- make the whole backend repeatable and trustworthy under presentation pressure

Steps:

1. Add a database reset or seed script.

2. Add a documented command sequence for starting the control node and generating traffic.
3. Write integration tests for enrollment, heartbeat, telemetry ingestion, filters, analytics, and SSE.
4. Verify that a failed session still aggregates correctly in both session detail and stats.
5. Verify that the system recovers cleanly if the dashboard or control node is restarted.
6. Fix any telemetry fields that are inconsistent or awkward for the frontend.

Minimum test coverage targets:

- enroll runner
- authenticate token
- update last-seen status
- create and update session rows
- persist telemetry events
- return filtered data
- stream new events

Outputs:

- repeatable demo setup
- backend regression coverage on the riskiest paths
- stable developer commands for rehearsal

Phase 6: Freeze Scope And Rehearse The Demo Path

Objective:

- stop feature churn and make the presentation reliable

Steps:

1. Freeze risky feature additions after the core demo path works.
2. Tune scenario timing so the UI updates at a presenter-friendly pace.
3. Rehearse the exact commands used on demo day.
4. Confirm the dashboard receives enough data to make charts and alerts look intentional.
5. Keep one fallback scenario ready in case the primary scenario is too noisy or too slow.
6. Document the startup order, reset steps, and recovery steps in one short runbook.

Recommended rehearsal script:

1. start Postgres
2. push schema
3. start control node

4. start dashboard
5. run `mixed-fleet`
6. run `failure-burst`
7. click through session detail

Definition of done:

- the backend team can recreate the demo state on demand without improvising

Week-By-Week Plan

Week 1: Foundations

Goals:

- finalize telemetry fields and failure categories
- add runner labels support
- add query filtering to runners, sessions, and events
- design the demo scenarios

Exit criteria:

- frontend receives stable API contracts
- at least one student can generate realistic demo traffic locally

Week 2: Real-Time And Analytics

Goals:

- implement SSE stream
- build analytics endpoints
- expand demo runner into multi-runner scenarios
- add background heartbeat scheduling

Exit criteria:

- live dashboard updates work end to end
- at least one chart can be fed from a real aggregate endpoint

Week 3: Reliability And Polish

Goals:

- harden scenario generation
- add tests for critical routes
- stabilize error handling and reconnect logic
- prepare seed/reset commands

Exit criteria:

- demo can be rerun from a clean database in under ten minutes
- backend can survive normal demo mistakes

Final 3-4 Days: Freeze And Rehearse

Goals:

- stop adding risky features
- tune the scenario data to look good in the UI
- verify timing of event bursts for the presentation
- document the operator flow for the presenter

Exit criteria:

- every backend endpoint used in the demo has been exercised in rehearsal
- the team knows exactly which commands to run and in what order

Non-Goals

Do not burn the schedule on these:

- gRPC
- mTLS
- multi-tenant RBAC
- distributed message brokers
- production-grade alerting
- full orchestration or task delegation

Those are good roadmap items and bad three-week demo items.

Backend Definition Of Done

The backend team is done when all of the following are true:

- multiple runners can report concurrently
- heartbeats happen automatically
- the control node supports filtering and aggregates
- the dashboard can consume a live SSE stream
- failure scenarios are intentional and explainable
- the demo can be reset and replayed on demand

If those six things work, the frontend team can make the system look excellent on stage.